FOR COMPLETE BEGINNERS

One class, many objects.

Complete Modern Java, episode 16: classes and objects, in Java and TypeScript

> Make your own type, see what Java fills in when you forget, and give it a constructor and a method.

A product has a name and a price. Four loose variables cannot say which name goes with which price.

A class describes the kind of thing; new makes an object; each object keeps its own values.

In this episode

- A class with fields
- new and objects
- Field defaults: 0 and null
- A constructor and this
- A method on the object
- An array of objects
- TypeScript classes

Coming next

- Protecting the fields (episode 17)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each file printed

Fields.java: new Product(), fields not set	`null, 0`
Fields.java: after setting the name	`Bread`
Local.java: a local read before a value	`variable count might not have been initialized`
Shop.java: bread.label(), milk.label()	`Bread: 150 cents, Milk: 99 cents`
Shop.java: for-each over a Product[]	`249`
shop.ts	`Bread: 150 cents, Milk: 99 cents`

Every row was run, not predicted.

02 Fields, and their defaults

A field that was never given a value starts with a default: 0 for a number, null for a String (no text yet).

A local variable gets no default: Java refuses to read it before it has a value.

Fields.java

```
class Product {
    String name;
    int priceCents;
}

void main() {
    Product bread = new Product();
    IO.println(bread.name);          // null
    IO.println(bread.priceCents);    // 0
    bread.name = "Bread";
    bread.priceCents = 150;
    IO.println(bread.name);          // Bread
}
```

> Printed null, 0, then Bread. Local.java stopped: variable count might not have been initialized.
>
> Run on a Mac, 2026-11-06: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 A constructor and a method

this.name is the field of the object being made; name is the value passed in.

Fields hold the information; methods describe what the object can do. One class, two objects.

Shop.java

```
class Product {
    String name;
    int priceCents;
    Product(String name, int priceCents) {
        this.name = name;
        this.priceCents = priceCents;
    }
    String label() {
        return name + ": " + priceCents + " cents";
    }
}

void main() {
    Product bread = new Product("Bread", 150);
    Product milk = new Product("Milk", 99);
    IO.println(bread.label());   // Bread: 150 cents
    IO.println(milk.label());    // Milk: 99 cents
}
```

> Printed Bread: 150 cents and Milk: 99 cents; with a Product[] and a for-each, 249.
>
> Run on a Mac, 2026-11-06: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript classes

The same shape. The type comes after the name, the constructor is called constructor, and every field inside a method needs this.

shop.ts

```
class Product {
    name: string;
    priceCents: number;
    constructor(name: string, priceCents: number) {
        this.name = name;
        this.priceCents = priceCents;
    }
    label(): string {
        return this.name + ": " + this.priceCents + " cents";
    }
}
```

> npx tsc --noEmit was quiet; without this. before priceCents it reported TS2663: Cannot find name.
>
> Run on a Mac, 2026-11-06: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Make a class

1. One kind of thing? One class.
2. new makes an object; each keeps its own values.
3. A constructor sets the fields as the object is made.
4. TypeScript: this. before every field in a method.
5. Challenge: first.name = "Toast". What do both labels print?

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-06 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.